LAB 7: Fuzzing for Input Validation Bugs

Software Engineering Lab Report | Property-Based Testing with Hypothesis

Student Name:	Alakh Gupta	**SRN:**	PES2UG24CS051
Course:	Software Engineering (SE)	**Repository:**	L6_L7_47_fuel_station

1. Objective & Learning Outcomes

The objective of this lab is to explore **fuzz testing** as a dynamic security and quality assurance technique by identifying and hardening input validation bugs in a Python application using the **Hypothesis** property-based fuzzing library. Key outcomes include understanding automated edge-case generation, shrinking minimal failing inputs, and applying defensive programming techniques to prevent unhandled crashes.

2. Step 1: Installing Dependencies

The required testing libraries were installed using pip:
• pip install hypothesis
• pip install pytest

3. Step 2: Running the Buggy Application

The application script under test implements a fuel station service with three functions: parse_purchase(text), volume_average(volumes_csv), and payable_bill(text). The original buggy implementation assumes well-formed inputs and performs naive string splits and conversions without validation.

```
alakhgupta@Alakhs-MacBook-Air Lab 7 % python processor.py
==== Running parse_purchase ====
Enter grade:litres (e.g. diesel:25): diesel:25
Grade and Litres: ('Diesel', 25.0)

==== Running volume_average ====
Enter pump volumes in litres as CSV (e.g. 10,25,40): 10,25,40
Average Volume: 25

==== Running payable_bill ====
Enter bill|discount (e.g. 2500|100): 2500|100
Payable Bill: 2850.0

alakhgupta@Alakhs-MacBook-Air Lab 7 %
```

Figure 1: Screenshot of running the initial application with nominal valid inputs.

4. Step 3 & 4: Property-Based Fuzz Testing & Bug Discovery

In test_processor.py (and test_fuel_station.py), test properties were defined using the @given(st.text() | st.none()) decorator from Hypothesis to verify that none of the functions crash on unexpected or malformed inputs.

```
alakhgupta@Alakhs-MacBook-Air Lab 7 % pytest test_processor.py
============================= test session starts ==============================
platform darwin -- Python 3.15.1, pytest-9.2.0, pluggy-1.7.0
rootdir: /Users/alakhgupta/Documents/L6_L7_47_fuel_station/Lab 7
plugins: hypothesis-6.170.15
collected 3 items

test_processor.py FFF                                                    [100%]

=================================== FAILURES ===================================
_____________________________ test_parse_purchase_no_crash _____________________________
  + ExceptionGroup: Hypothesis found 3 distinct failures.
    ---------------- 1 ----------------
    AssertionError: parse_purchase crashed with: could not convert string to float: ''
    Falsifying example: test_parse_purchase_no_crash(s=':')
    ---------------- 2 ----------------
    AssertionError: parse_purchase crashed with: not enough values to unpack (expected 2, got 1)
    Falsifying example: test_parse_purchase_no_crash(s='')
    ---------------- 3 ----------------
    AssertionError: parse_purchase crashed with: 'NoneType' object has no attribute 'split'
    Falsifying example: test_parse_purchase_no_crash(s=None)

_____________________________ test_volume_average_no_crash _____________________________
  + ExceptionGroup: Hypothesis found 2 distinct failures.
    ---------------- 1 ----------------
    AssertionError: volume_average crashed with: 'NoneType' object has no attribute 'split'
    Falsifying example: test_volume_average_no_crash(s=None)
    ---------------- 2 ----------------
    AssertionError: volume_average crashed with: invalid literal for int() with base 10: ''
    Falsifying example: test_volume_average_no_crash(s='')

_____________________________ test_payable_bill_no_crash _____________________________
  + ExceptionGroup: Hypothesis found 3 distinct failures.
    ---------------- 1 ----------------
    AssertionError: payable_bill crashed with: could not convert string to float: ''
    Falsifying example: test_payable_bill_no_crash(s='|')
    ---------------- 2 ----------------
    AssertionError: payable_bill crashed with: not enough values to unpack (expected 2, got 1)
    Falsifying example: test_payable_bill_no_crash(s='')
    ---------------- 3 ----------------
    AssertionError: payable_bill crashed with: 'NoneType' object has no attribute 'split'
    Falsifying example: test_payable_bill_no_crash(s=None)

=========================== short test summary info ============================
FAILED test_processor.py::test_parse_purchase_no_crash - ExceptionGroup: Hypothesis found 3 distinct failures.
FAILED test_processor.py::test_volume_average_no_crash - ExceptionGroup: Hypothesis found 2 distinct failures.
FAILED test_processor.py::test_payable_bill_no_crash - ExceptionGroup: Hypothesis found 3 distinct failures.
============================== 3 failed in 0.70s ===============================

alakhgupta@Alakhs-MacBook-Air Lab 7 %
```

Figure 2: Screenshot of pytest failures exposed by Hypothesis fuzzing on buggy code.

Analysis of Failing Test Cases Discovered by Hypothesis:

Function	Falsifying Input	Exception / Bug Type	Root Cause
parse_purchase(text)	s = None s = '' s = ':'	AttributeError ValueError ValueError	NoneType has no split; empty string has 1 item (expected 2); empty litres cannot convert to float.
volume_average(csv)	s = None s = ''	AttributeError ValueError	NoneType has no split; int('') fails on empty CSV elements.
payable_bill(text)	s = None s = '' s = '|'	AttributeError ValueError ValueError	NoneType has no split; splitting empty string lacks delimiter; empty bill cannot convert to float.

5. Step 5: Defensive Hardening & Application Fixes

Defensive programming principles were applied across all three functions to handle edge cases gracefully:
• **Type Validation:** Verified that incoming arguments are valid string instances using isinstance(..., str), returning safe defaults when None or non-strings are passed.
• **Structure & Length Checks:** Verified that delimiter splitting produces exactly two non-empty tokens before unpacking.
• **Number Formatting & Boundary Checks:** Protected numeric parsing with try...except (ValueError, OverflowError) and checked against math.isnan, math.isinf, and negative values.
• **Safe Fallbacks:** Returned None for invalid purchases, 0 for empty/invalid volume CSVs, and 0.0 for invalid bills.

```
alakhgupta@Alakhs-MacBook-Air Lab 7 % python processor.py
==== Running parse_purchase ====
Enter grade:litres (e.g. diesel:25): diesel:25
Grade and Litres: ('Diesel', 25.0)

==== Running volume_average ====
Enter pump volumes in litres as CSV (e.g. 10,25,40): 10,25,40
Average Volume: 25

==== Running payable_bill ====
Enter bill|discount (e.g. 2500|100): 2500|100
Payable Bill: 2850.0

alakhgupta@Alakhs-MacBook-Air Lab 7 %
```

Figure 3: Screenshot of running the hardened application script successfully.

6. Step 6: Re-Running Tests on Hardened Application

After hardening the application script, pytest -v test_processor.py was re-executed. Hypothesis generated hundreds of arbitrary Unicode strings, control characters, empty strings, and None values. All test cases passed cleanly with **zero crashes**.

```
alakhgupta@Alakhs-MacBook-Air Lab 7 % pytest -v test_processor.py
============================= test session starts ==============================
platform darwin -- Python 3.15.1, pytest-9.2.0, pluggy-1.7.0 -- /opt/homebrew/opt/python@3.15/bin/python3.15
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /Users/alakhgupta/Documents/L6_L7_47_fuel_station/Lab 7
plugins: hypothesis-6.170.15
collected 3 items

test_processor.py::test_parse_purchase_no_crash PASSED                   [ 33%]
test_processor.py::test_volume_average_no_crash PASSED                   [ 66%]
test_processor.py::test_payable_bill_no_crash PASSED                     [100%]

============================== 3 passed in 0.70s ===============================

alakhgupta@Alakhs-MacBook-Air Lab 7 %
```

Figure 4: Screenshot of pytest showing all Hypothesis property tests passing (100%).

7. Summary of Deliverables

Deliverable	Artifact Location	Status
1. PDF Document with Screenshots	Lab 7/PES2UG24CS051_LAB07.pdf	Completed
2. Fixed Application Python Script	Lab 7/fuel_station.py & processor.py	Hardened & Verified
3. Updated Test Script	Lab 7/test_fuel_station.py & test_processor.py	All 3 Tests Passing